

```

import numpy as np
from scipy.special import erfc
import matplotlib.pyplot as plt

def q_func(x):
    return 0.5 * erfc(x / np.sqrt(2))

def text_to_bits(text, bits_per_symbol):
    bit_string = "".join(f"{ord(ch):08b}" for ch in text)
    bit_array = np.array([int(bit) for bit in bit_string], dtype=int)
    remainder = len(bit_array) % bits_per_symbol
    if remainder != 0:
        padding = bits_per_symbol - remainder
        bit_array = np.concatenate([bit_array, np.zeros(padding, dtype=int)])
    return bit_array

def bits_to_index(triad):
    return int(triad[0] * 4 + triad[1] * 2 + triad[2])

def add_noise(signal, variance, rng):
    noise = rng.normal(0.0, np.sqrt(variance), size=signal.shape)
    return signal + noise

def decide_symbols(received, constellation_levels):
    distances = np.abs(received[:, None] - constellation_levels[None, :])
    return np.argmin(distances, axis=1)

def simulate_ser(symbols, indices, variance, rng, repetitions, constellation_levels):
    :
    total_symbols = len(symbols) * repetitions
    error_count = 0
    for _ in range(repetitions):
        received_symbols = add_noise(symbols, variance, rng)
        detected_indices = decide_symbols(received_symbols, constellation_levels)
        error_count += np.sum(detected_indices != indices)
    return error_count / total_symbols

def compute_theoretical_ser(Eb, variances, modulation_order):
    variances_array = np.asarray(variances, dtype=float)
    ebno_values = Eb / (2.0 * variances_array)
    decision_factor = np.sqrt(
        (6 * np.log2(modulation_order) / (modulation_order**2 - 1)) * ebno_values
    )
    ser = 2 * (modulation_order - 1) / modulation_order * q_func(decision_factor)
    return ser, ebno_values

def main():
    constellation_levels = np.array([-7, -5, -3, -1, 1, 3, 5, 7], dtype=float)
    bits_per_symbol = 3
    noise_variances = [0.1, 0.5, 1.0, 2.0]
    message_text = "BITS PILANI"
    monte_carlo_rounds = 2000

    rng = np.random.default_rng(2025)

    bit_array = text_to_bits(message_text, bits_per_symbol)
    symbol_count = len(bit_array) // bits_per_symbol
    symbols = np.empty(symbol_count, dtype=float)
    indices = np.empty(symbol_count, dtype=int)

    # Loop explicitly transmits each 3-bit word using its 8-PAM level, matching task
    instructions.
    for symbol_idx in range(symbol_count):
        start = symbol_idx * bits_per_symbol
        triad = bit_array[start : start + bits_per_symbol]

```

```
    idx = bits_to_index(triad)
    indices[symbol_idx] = idx
    symbols[symbol_idx] = constellation_levels[idx]

ser_sim = []
eb = np.mean(constellation_levels**2) / bits_per_symbol
ebno_lin = []

for variance in noise_variances:
    ser = simulate_ser(
        symbols,
        indices,
        variance,
        rng,
        monte_carlo_rounds,
        constellation_levels,
    )
    ser_sim.append(ser)
    ebno_lin.append(eb / (2.0 * variance))

ser_sim = np.array(ser_sim)
ebno_lin = np.array(ebno_lin)

ser_theo, ebno_theo = compute_theoretical_ser(
    eb, noise_variances, len(constellation_levels)
)

ebno_dB = 10 * np.log10(ebno_lin)

plt.figure(figsize=(8, 5))
plt.semilogy(ebno_dB, ser_sim, "bo-", label="Simulated")
plt.semilogy(10 * np.log10(ebno_theo), ser_theo, "r^-", label="Theoretical")
plt.grid(True, which="both", ls=":")
plt.legend()
plt.tight_layout()
plt.show()

if __name__ == "__main__":
    main()
```